

AI Security 2.3. AI Threat Modelling

Task 1: Introduction

AI isn't something organisations are still waiting on; it's already embedded in enterprise operations. Language models handle customer support tickets. Recommendation engines surface products to millions of users. Fraud detection systems make real-time decisions that affect people's lives. Behind every one of these deployments is an attack surface that most security teams have never been trained to assess.

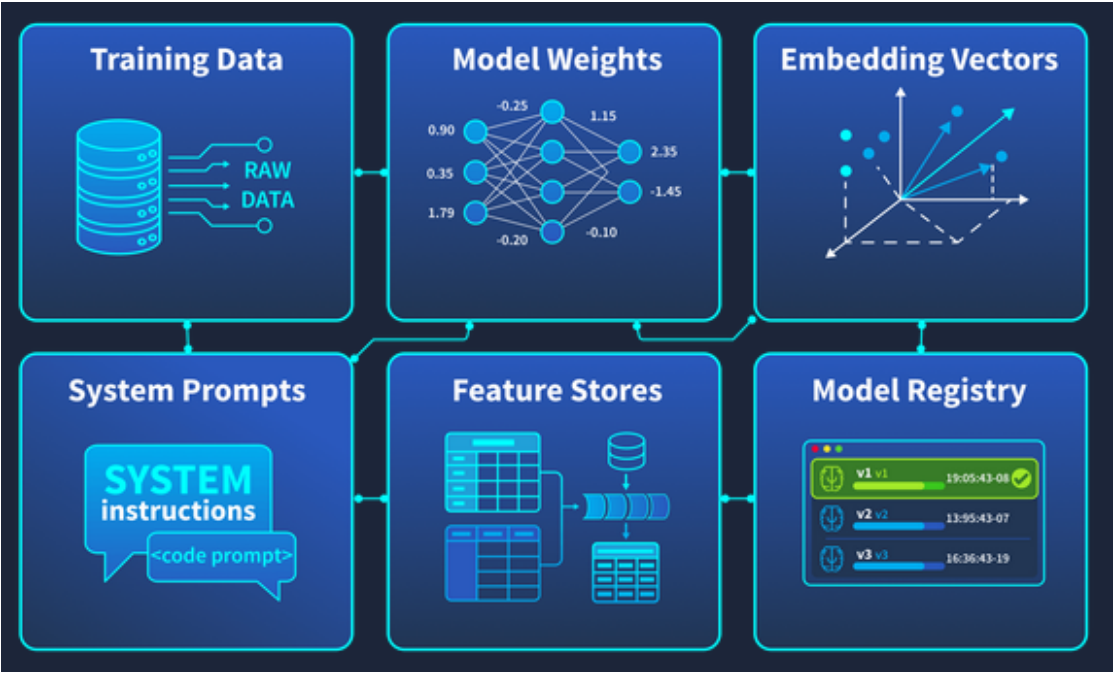
Traditional threat modelling provides a strong foundation, and frameworks like STRIDE have helped defenders systematically identify security threats for over two decades. But AI systems introduce **assets, behaviours, and failure modes** that those frameworks weren't designed to handle. Training data can be poisoned. Model weights can be stolen. Prompts can be injected. And the outputs? They're non-deterministic, meaning the same system can behave differently each time it's queried.

Learning Objectives

- Identify AI specific assets and attack surfaces that don't exist in traditional applications
- Apply STRIDE threat categories to AI/ML system components with appropriate context
- Use MITRE ATLAS to enumerate adversarial techniques targeting AI systems
- Map OWASP LLM Top 10 risks to architectural components to identify where threats live and how to prioritise them
- Produce a structured threat assessment for an AI deployment

Task 2: AI-Specific Assets and Attack Surfaces

If you have threat modelled traditional applications before, you are used to thinking about a familiar set of assets: databases, source code, configuration files, API keys, and user credentials. You know what they are, where they live, and how to protect them. AI systems change the picture. They introduce an entirely new class of assets that most security teams have never had to inventory, classify, or defend. Missing these assets during a threat assessment means missing entire categories of risk, and that's exactly the gap attackers exploit.



AI Assets You Need to Know

Asset	What It Is	Why It Matters
Training Data	The datasets used to teach the model its behaviour	Poisoning this data corrupts the model's outputs at the source. Unlike a database compromise, the damage is baked into the model itself.
Model Weights / Parameters	The numerical values that define what the model has learned	These <i>are</i> the model. Stealing them means an attacker has a functional copy of your AI , months of compute and potentially millions in investment, gone.

Asset	What It Is	Why It Matters
Embedding Vectors	Numerical representations of text or data used for similarity computation, retrieval, or as input features to downstream models	Used in RAG pipeline, recommendation engines, and fraud detection systems. Poisoning or manipulating embeddings alters what information models see at query time.
System Prompts	Instructions that define the model's behaviour, constraints, and persona	Leaking these reveals your security controls, business logic, and guardrails, giving attackers a roadmap to bypass them.
Feature Stores	Preprocessed data repositories that feed real-time model inputs	Tampering with features changes what the model "sees" at inference time, without touching the model itself.
Model Registry / Artifacts	Stored versions of trained models ready for deployment	A compromised registry means an attacker can swap a legitimate model for a backdoored one, and no one may notice until it's too late.

None of these assets map neatly onto traditional asset categories. A stolen database is serious, but a stolen model is a fundamentally different kind of loss, you can't just rotate a credential and move on. The asset that defines the model's learned behaviour is its model weights; once those are exfiltrated, the attacker has a functional copy of your AI. Meanwhile, if an attacker wants to give themselves a roadmap of your LLM 's security controls and behavioural constraints, the asset they would target is your **system prompts**. And a poisoned training data set doesn't trigger the same alerts as a modified database record, because the corruption only surfaces after the model has been retrained and redeployed.

Beyond new asset types, AI systems also behave differently from traditional software, affecting how we model threats. Two characteristics worth noting:

- **Non-deterministic behaviour:** AI models, especially LLMs, can produce different outputs for the same input. This makes testing, auditing, and incident reproduction significantly harder than with deterministic software. If you've completed earlier rooms in this path, you'll already be familiar with this concept.
- **The black box problem:** Most AI models, particularly deep neural networks, lack the explainability of traditional application logic. You can't step through a model's reasoning the way you'd trace a code path. This forces defenders to think in terms of input-output behaviour and failure modes rather than code-level inspection.

Both of these characteristics have direct implications for threat modelling, and we will see them repeatedly surface as we work through the frameworks in upcoming tasks. For now, the key takeaway is simple: **AI systems aren't just traditional applications with a model bolted on.**

They have different assets, behaviours, and ways of failing, and our threat models need to account for all of it.

In a RAG-based system, which AI asset type is used to retrieve relevant context at query time?

Embedding Vectors

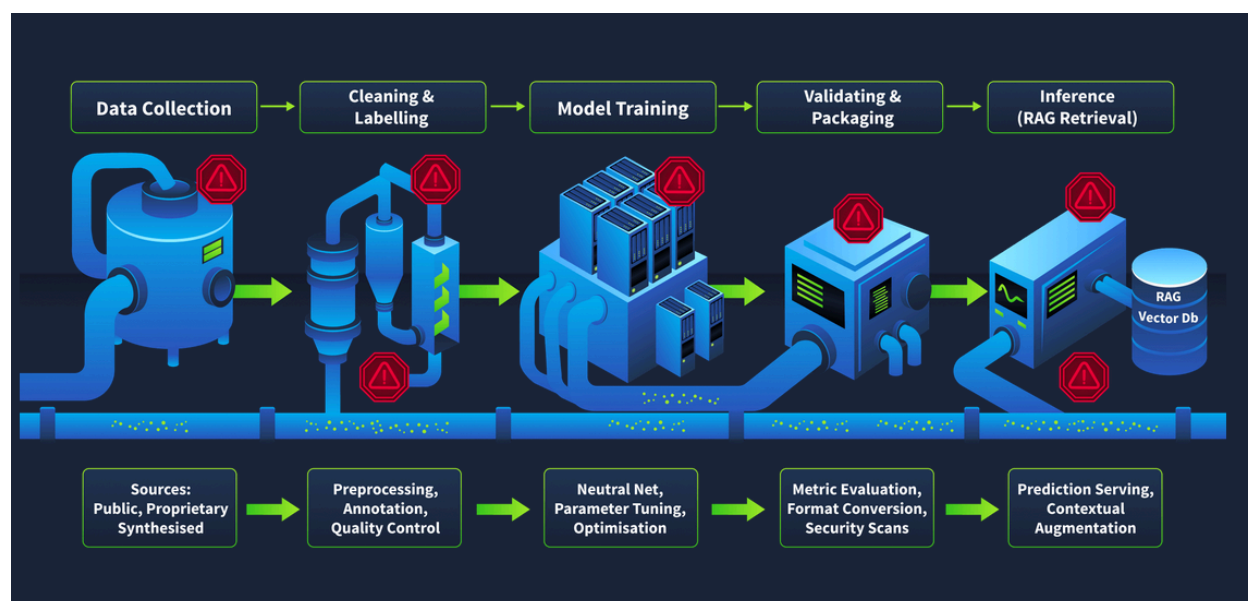
An attacker gains access to MegaCorp's model registry and swaps the production model for a modified version. Which AI-specific asset has been compromised?

Model Registry / Artifacts

Task 3: Data Supply Chain and STRIDE's Gaps

The AI Data Supply Chain: Traditional applications have software supply chains, dependencies, libraries, container images. You have likely already encountered supply chain threats in the form of compromised packages or malicious

dependencies. AI systems inherit all of those risks and add an entirely separate supply chain built around data.



Stage 1: Data Collection: Training data is gathered from multiple sources, including web scraping, purchased datasets, internal databases, user-generated content, and third-party providers. At this stage, an attacker who can contribute or influence any of these sources has a foothold.

Stage 2: Cleaning and Labelling: Raw data is preprocessed, filtered, and labelled. In some pipelines this involves external annotation teams or automated labelling tools. In other cases, such as fraud detection, labels are derived implicitly from outcomes, like chargebacks or investigation results. Regardless of the method, compromised labels lead the model to learn the wrong associations. A mislabelled dataset doesn't look corrupted. It just quietly teaches the model to make incorrect decisions.

Stage 3: Model Training: The model learns patterns from the prepared data over days or weeks of compute. Any poison that survived the first two stages is now embedded in the model's weights. Unlike a compromised library you can patch, a poisoned model may need to be retrained from scratch, at significant time and cost.

Stage 4: Validation and Packaging: The trained model is evaluated, versioned, and stored in a model registry for deployment. If the registry itself is compromised, an

attacker can swap a validated model for a backdoored one. The backdoored model passes standard validation checks because the trigger inputs (the specific patterns that activate the malicious behaviour) are absent from the validation dataset. Everything looks clean until the model encounters those triggers in production.

Stage 5: Inference: The model serves predictions in production. For LLM-based systems, this stage often includes a retrieval pipeline that retrieves additional context from vector databases or document stores at query time, introducing yet another injection point that doesn't exist in traditional applications.

Each stage is a link in the chain, and each link is a potential point of compromise. The critical difference from traditional software supply chains is time. A compromised npm package can be detected and reverted within hours. A poisoned training dataset may not reveal its effects for weeks or months, only surfacing after the model is retrained, validated, and deployed to production.

Why STRIDE Alone Falls Short

Now that we understand AI's new assets and new supply chain concept, let's address the framework question: can we just use STRIDE as-is?

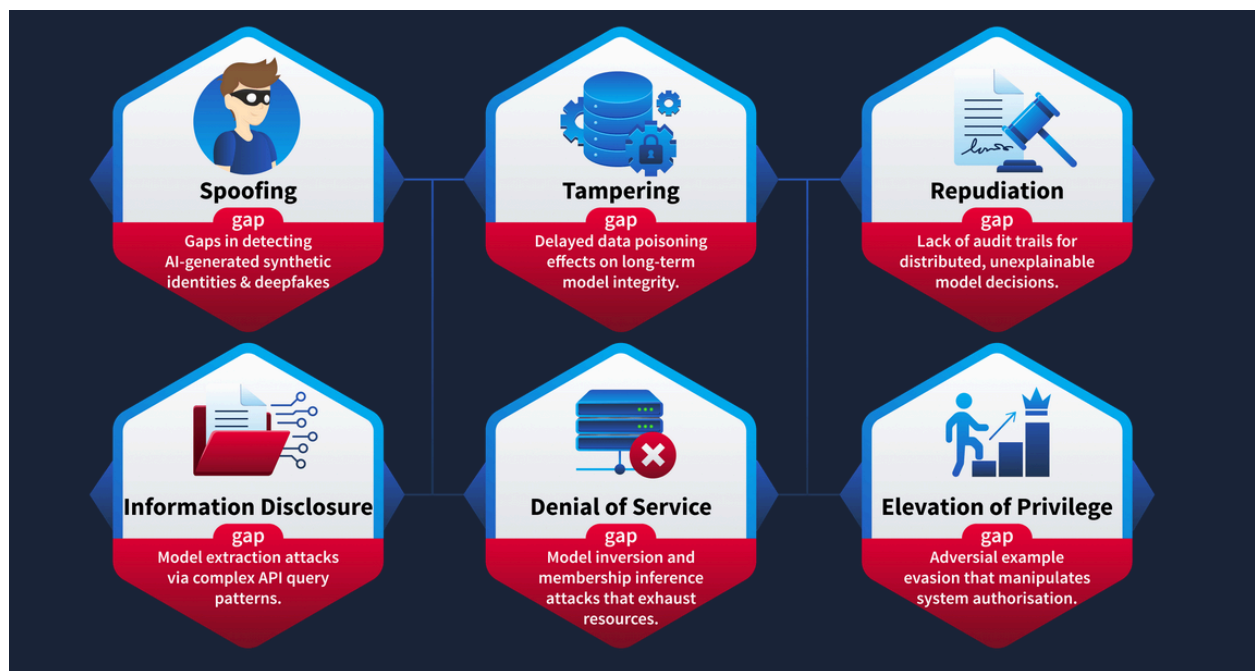
STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), has been the backbone of threat modeling since Microsoft introduced it in the late 1990s. It remains highly effective for traditional applications. But when applied to AI systems without adaptation, it has documented gaps:

Data integrity isn't a first-class concern at the training level. STRIDE's Tampering category works well for data in transit or at rest. But tampering with training data is fundamentally different, the effects are diffuse, delayed, and nearly invisible. A poisoned training set doesn't throw an error. It produces a model that behaves incorrectly in subtle, hard-to-detect ways.

Adversarial manipulation of model behaviour doesn't fit neatly into one category. Crafting inputs designed to make a model misclassify, hallucinate, or bypass safety guardrails spans multiple STRIDE categories simultaneously, it's part Tampering, part Spoofing, part Elevation of Privilege depending on context. STRIDE wasn't designed for threats that blur across categories this way.

The scope of privilege has expanded beyond what STRIDE originally envisioned. When a model can take actions, browse the web, execute code, send emails, query databases, the Elevation of Privilege category still applies, but what constitutes "privilege" is fundamentally broader. A jailbroken chatbot with tool access isn't just a traditional privilege escalation. The model's entire set of tool permissions becomes the attacker's capabilities.

Model-specific intellectual property theft is a different kind of disclosure. Extracting a model's weights through carefully crafted API queries is technically Information Disclosure, but it's profoundly different from exfiltrating a database. The stolen asset is the organisation's entire AI capability, not a dataset, but a trained intelligence.



An attacker injects crafted data points into a training pipeline over several months, gradually shifting the model's decision boundaries. At which supply chain stage does the attacker inject the malicious data?

Data collection

Which STRIDE category is insufficient for capturing the delayed, diffuse effects of training data poisoning?

Tampering

Task 4: Adapting STRIDE for AI Systems

STRIDE Refresher



Threat Category	Security Property Violated	Traditional Meaning
S — Spoofing	Authenticity	Pretending to be someone or something you're not
T — Tampering	Integrity	Modifying data or code without authorisation
R — Repudiation	Non-repudiability	Denying that you performed an action
I — Information Disclosure	Confidentiality	Exposing information to unauthorised parties
D — Denial of Service	Availability	Making a system or resource unavailable
E — Elevation of Privilege	Authorisation	Gaining access or capabilities beyond what's permitted

1. S — Spoofing: Data Source Impersonation

Traditional: An attacker forges credentials to impersonate a legitimate user or service.

Primary AI Manifestation → Data Source Impersonation. In RAG architectures, the model retrieves context from external sources, vector databases, document stores, and web content and treats that context as trustworthy. An attacker who can inject content into these sources effectively spoofs the knowledge the model

relies on, causing it to generate responses grounded in attacker-controlled information. Other AI-related spoofing threats include:

- **Model impersonation:** deploying a look-alike API endpoint that mimics a legitimate AI service
- **Adversarial identity attacks:** crafting inputs that fool AI-based identity verification systems (facial recognition, voice auth)

2. T — Tampering: Data Poisoning

Traditional: An attacker modifies data in transit or at rest, altering database records, intercepting API responses, changing configuration files.

Primary AI Manifestation → Data Poisoning. An attacker injects malicious data into the training pipeline, causing the model to learn incorrect patterns. Unlike traditional tampering, the effects are delayed, they're embedded during training and only surface during inference. Poisoning can be targeted (forcing specific misclassifications) or untargeted (degrading overall performance).

Other AI-related tampering threats include:

- **Model manipulation:** directly modifying model weights in storage or swapping models in the registry with backdoored versions
- **Prompt injection:** manipulating instructions or context the model receives at inference time (direct or indirect). Note that prompt injection's STRIDE classification is context-dependent: it maps to **Tampering** when the attacker is altering the model's input, but can also manifest as **Elevation of Privilege** when the goal is bypassing guardrails
- **Feature manipulation:** altering input features so the model makes decisions based on tampered data

3. R — Repudiation: Unexplainable Model Decisions

Traditional: A user performs an action and later denies it because the system lacks adequate logging or audit trails.

Primary AI Manifestation → Lack of Decision Audit Trails. When an AI model makes a consequential decision, approves a loan, flags a transaction, or denies a

claim, can you trace why? Most ML models lack built-in explainability. Without robust logging of inputs, outputs, model versions, and retrieval context, reproducing or explaining a specific decision after the fact is extremely difficult.

Other AI-related repudiation threats include:

- **Prompt and context volatility:** the full context behind an LLM output (system prompt, user input, RAG context, conversation history, temperature) is rarely captured completely
- **Model version ambiguity:** without deployment logs, you can't attribute a specific output to a specific model state

4. I — Information Disclosure: Model Extraction

Traditional: Sensitive data is exposed through data breaches, insecure APIs, verbose error messages, or improper access controls.

Primary AI Manifestation → Model Extraction (Model Stealing). An attacker systematically queries a model's API and uses the input-output pairs to reconstruct a functionally equivalent copy of the model. This requires no access to the model's internals; only its public-facing endpoint is needed. The stolen model represents significant intellectual property loss and can be probed offline for adversarial weaknesses.

Other AI-related information disclosure threats include:

- **Training data extraction:** crafting queries that cause the model to regurgitate memorised training data, potentially including PII or proprietary content
- **System prompt leakage:** using prompt extraction techniques to reveal internal instructions, guardrails, and business logic
- **Embedding inversion:** reversing embedding vectors to reconstruct the original source documents from a vector database

5. D — Denial of Service: Inference Cost Exploitation

Traditional: Flooding a system with traffic to exhaust resources and make the service unavailable.

Primary AI Manifestation → Inference Cost Exploitation (Denial of Wallet). AI inference is orders of magnitude more expensive than traditional API calls. In cloud-based deployments billed per token or per query, an attacker can inflict financial damage without taking the system offline. By generating large volumes of expensive queries, long prompts, requests for maximum-length outputs, they drive operational costs to unsustainable levels.

Other AI-related denial of service threats include:

- **GPU resource exhaustion:** high-volume or complex queries that saturate compute capacity, queuing or dropping legitimate requests
- **Sponge examples:** adversarial inputs crafted to maximise the computational resources consumed during a single inference call
- **Training pipeline disruption:** injecting massive volumes of junk data to delay or corrupt retraining cycles

6. E — Elevation of Privilege: Jailbreaking and Excessive Agency

Traditional: Gaining higher-level access or capabilities than intended, an unprivileged user getting admin access, a service account performing unauthorised actions.

Primary AI Manifestation → Jailbreaking / Guardrail Bypass. An attacker crafts prompts that cause an LLM to ignore its safety guidelines, content policies, or behavioural restrictions. The model is designed to refuse certain requests, but the attacker's input "elevates" their access to capabilities the model was instructed to restrict. This is conceptually similar to privilege escalation, the attacker doesn't get root on a server, but they gain unrestricted access to the model's full capabilities.

Other AI-related elevation of privilege threats include:

- **Excessive agency:** when an AI system's tool permissions exceed what's appropriate for its context, turning a chatbot compromise into access to internal databases, email systems, or code execution
- **Tool use exploitation:** manipulating an agentic AI into using its tools (web browsing, file writing, API calls) for unintended purposes

- **Cross-plugin escalation:** compromising one plugin's input to affect the model's behaviour with other, more privileged plugins

What STRIDE Still Misses

Even with these adaptations, some AI threats don't map cleanly to any single STRIDE category:

Adversarial examples: inputs designed to cause misclassification, span Tampering, Spoofing, and Elevation of Privilege depending on context. There's no single lens that captures them fully.

Model bias and fairness issues are security-adjacent concerns with real regulatory and compliance implications, but they don't fit traditional threat categories. A biased model isn't being "attacked", it's failing in a way STRIDE wasn't designed to describe.

Emergent behaviours in large models, capabilities or behaviours that weren't explicitly trained for and may not be anticipated, are a class of risk with no traditional parallel. You can't threat model behaviour that nobody predicted would exist.

STRIDE Consolidated Mapping:

STRIDE Category	Primary AI Manifestation	Other AI Threats	MegaCorp Example
Spoofing	Data source impersonation (RAG injection)	Model impersonation, adversarial identity attacks	Fake policy docs injected into chatbot knowledge base
Tampering	Data Poisoning	Model manipulation, prompt injection, feature tampering	Crafted transactions shift fraud model's decision boundaries
Repudiation	Lack of decision audit trails	Context volatility, model version ambiguity	Can't explain why fraud model approved a suspicious transaction
Info Disclosure	Model extraction / stealing	Training data extraction, prompt leakage, embedding inversion	Competitor reconstructs recommendation engine via API queries

STRIDE Category	Primary AI Manifestation	Other AI Threats	MegaCorp Example
Denial of Service	Inference cost exploitation (denial of wallet)	GPU exhaustion, sponge examples, pipeline disruption	Chatbot API flooded with expensive prompts; bill (\$) spikes 12x
Elevation of Privilege	Jailbreaking / guardrail bypass	Excessive agency, tool exploitation, cross-plugin escalation	Jailbroken chatbot used to query customer PII via database tools

What is the primary AI-specific manifestation of Information Disclosure in the STRIDE-AI mapping?

Model Extraction

An attacker crafts prompts that cause an LLM to bypass its safety guidelines and content restrictions. Which STRIDE category does this map to?

Elevation of Privilege

Which OWASP LLM Top 10 (2025) entry addresses the risks of AI systems being granted too many permissions or too much autonomy?

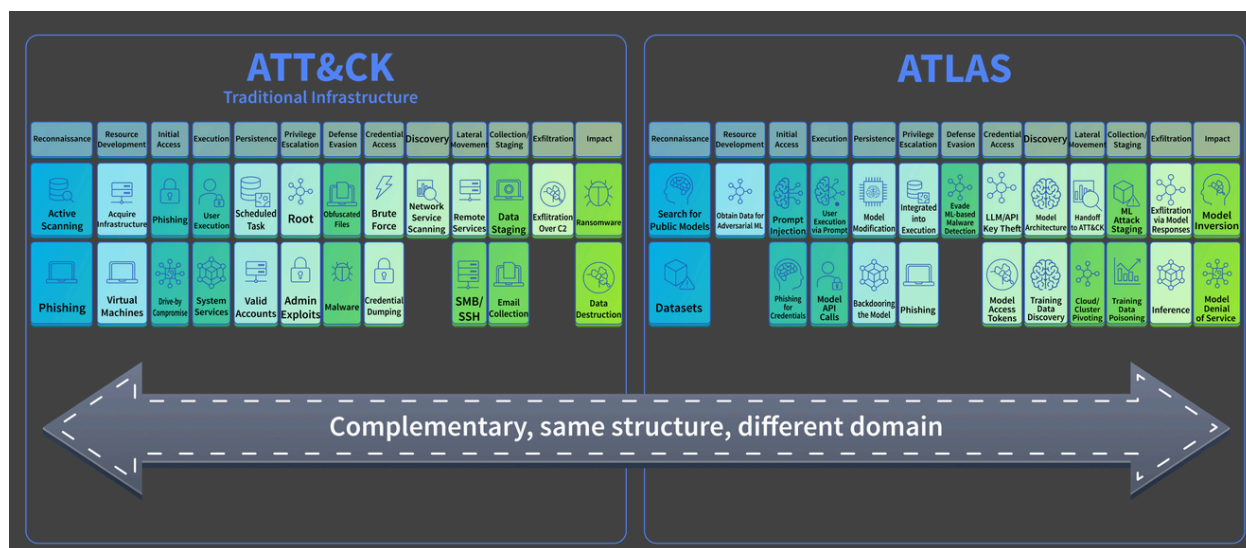
LLM06:2025 — Excessive Agency

An attacker drives your monthly inference bill from \$15,000 to \$180,000 without taking your service offline. What is this type of attack commonly called?

Denial of Wallet

Task 5: MITRE ATLAS: The AI Threat Technique Catalogue

ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) is a knowledge base of adversary tactics and techniques targeting AI and ML systems. Think of it as **MITRE ATT&CK's AI-focused counterpart**. If you've used [MITRE ATT&CK \(opens in new tab\)](#) to map adversary behaviour against traditional infrastructure, ATLAS gives you the same structured approach for AI systems.



How Is Structured:

ATLAS follows the same hierarchy you already know from **MITRE ATT&CK**:

Component	What It Answers	Example
Tactic	<i>Why</i> the adversary's goal	ML Attack Staging (AML.TA0012)
Technique	<i>How</i> the method used to achieve it	DATA Poisoning (AML.T0020)
Sub-technique	<i>Specifically how</i> a variant of the method	Craft Adversarial Data (AML.T0043.004)
Mitigation	<i>What stops it</i> the defensive countermeasure	Input validation, data provenance tracking

Tactics are the columns of the ATLAS matrix. Techniques sit within those columns. When you are threat modeling, you start with a tactic (what the attacker wants to achieve) and drill into techniques (how they'd achieve it against your specific system).



Key Techniques You Need to Know:

Data Poisoning (AML.T0020): Injecting malicious data into training pipelines to corrupt model behaviour. Effects are delayed and persist until the model is retrained on clean data. Maps to STRIDE: Tampering

Model Extraction (AML.T0024): Systematically querying a model's API to reconstruct a functional copy. Requires no internal access, just the public endpoint and enough queries. Maps to STRIDE: Information Disclosure.

Evade ML Model (AML.T0015): Crafting adversarial data that prevents a model from correctly identifying the contents of the input. This threat spans multiple STRIDE categories simultaneously, Tampering, Spoofing, and Elevation of Privilege, depending on context. Adversaries may use this to evade malware detection, bypass content filters, or cause misclassification in downstream tasks.

LLM Prompt Injection (AML.T0051): Manipulating an LLM's behaviour by injecting instructions through direct user input or indirect content the model processes. The distinction matters: direct injection is a user crafting malicious input in the chat interface, while indirect injection is malicious instructions embedded in content

the model retrieves or processes (such as documents in a RAG pipeline). For MegaCorp, indirect injection via RAG the knowledge base is the primary vector.

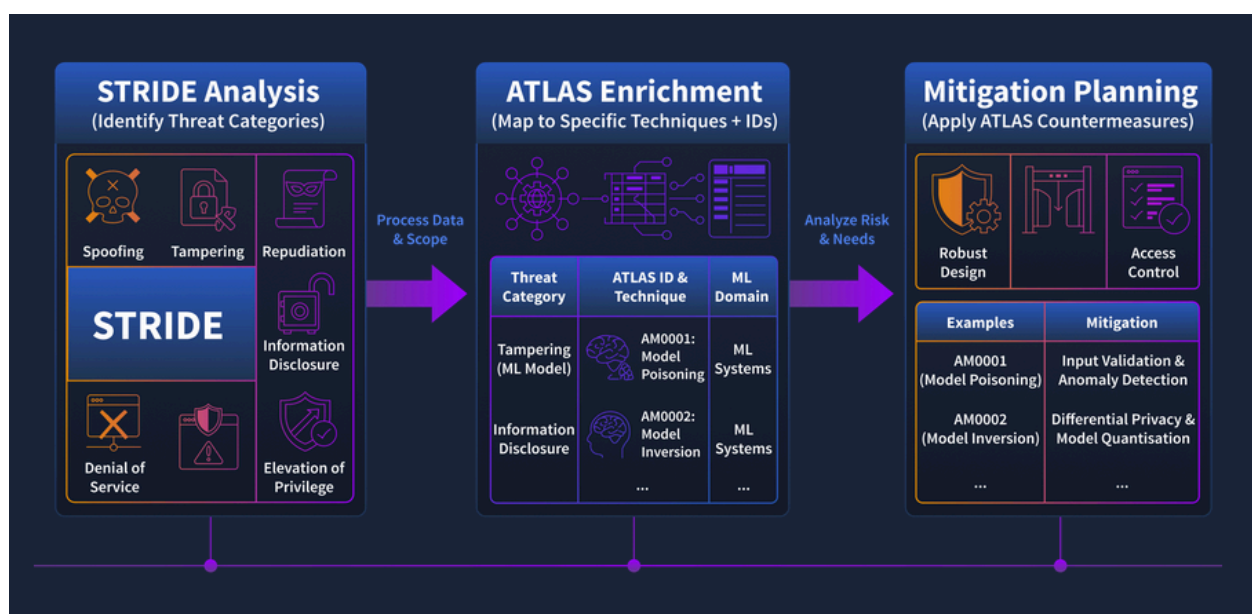
Maps to STRIDE: Tampering

Backdoor ML Model (AML.T0018): Embedding hidden triggers in a model during training. The model performs normally on standard inputs but behaves maliciously when a specific trigger pattern is present. Think of it as a logic bomb, but inside a neural network.

Using ATLAS During Threat Modeling:

ATLAS isn't a replacement for STRIDE, it's the enrichment layer. Here's how the two work together in practice:

- **Start with STRIDE:** Walk each AI component through the six threat categories to identify "*what could go wrong*"
- **Enrich with ATLAS:** For each identified threat, look up the corresponding ATLAS technique to get the specific *how*, including documented attack methods and real-world case studies
- **Apply mitigations:** ATLAS provides recommended countermeasures for each technique, giving you actionable defensive guidance



Real-World Case Studies

Atlas includes 52 documented case studies (opens in new tab) of real AI attacks. Two worth noting:

ShadowRay (opens in new tab) (AML.CS0023): Attackers exploited vulnerabilities in Ray, a popular framework for distributed AI workloads, to compromise AI training infrastructure in the wild. This demonstrated that AI supply chain attacks aren't theoretical, they're happening against production systems.

Morris II Worm (opens in new tab) (AML.CS0024): Researchers demonstrated a self-replicating prompt injection worm that could spread between AI agents through RAG-based email systems. The worm injected its payload into the model's context without user interaction, extracted PII, and automatically propagated it to other agents.

Both cases are documented in ATLAS with full technique mappings, giving you a concrete reference for what these attacks look like in practice.

| What does the acronym ATLAS stand for?

Adversarial Threat Landscape for Artificial-Intelligence Systems

| Which ATLAS case study described a self-replicating prompt injection worm that spread between AI agents via RAG email systems?

Morris II

| What is the ATLAS technique ID for Model Extraction?

AML.T0024

Task 6: OWASP LLM Top 10: Mapping Risks to Components

The OWASP Top 10 for LLM Applications is a community-driven list of the most critical security risks specific to large language model deployments. Published by the OWASP GenAI Security Project, it's built from real-world incidents, researcher findings, and industry consensus.

The 2025 List With Component Mapping

The table below doesn't just list the ten risks; it also shows **where each risk lives** in a typical LLM architecture. This is what turns the OWASP Top 10 from a reference document into an actionable assessment tool.

#	Risk	What It Means	Where It Lives (Vulnerable Components)
LLM01	Prompt Injection	Attacker manipulates model behaviour through crafted inputs, direct or indirect	LLM inference endpoint (direct injection via user input), vector database / RAG pipeline (indirect injection via retrieved content), any component that feeds text to the model
LLM02	Sensitive Information Disclosure	Model outputs reveal PII, credentials, or proprietary data	LLM inference endpoint (model memorisation), training pipeline (sensitive data in training set), system prompt (credentials or logic embedded in prompt)
LLM03	Supply Chain	Compromised models, training data , plugins, or dependencies	Training pipeline (third-party datasets, compromised base models, poisoned fine-tuning data), model registry (models retrieved from external repos such as Hugging Face), plugin/tool integrations (vulnerable or compromised third-party dependencies)
LLM04	Data and Model Poising	Corrupted training data or model weights alter behaviour	Training pipeline (data injection point), model registry (model swap), feature store (manipulated input features)
LLM05	Improper Output Handling	LLM outputs aren't validated before downstream use	Web frontend (unsanitised output rendered in browser, XSS risk), API gateway (model output passed to downstream services without validation), any system consuming model responses
LLM06	Excessive Agency	LLM granted too many permissions, tools, or autonomy	LLM inference endpoint (where jailbreaking enables tool abuse), tool integrations (database query tools, code

#	Risk	What It Means	Where It Lives (Vulnerable Components)
			execution, email sending), API gateway (overly broad API permissions granted to the model), agentic orchestration layer
LLM07	System Prompt Leakage	Internal prompts containing sensitive logic or credentials are exposed	LLM inference endpoint (prompt extraction attacks), system prompt configuration (credentials or API keys stored in prompts rather than secure vaults)
LLM08	Vector and Embedding Weaknesses	Vulnerabilities in RAG systems, vector databases, and embeddings	Vector database (embedding poisoning, similarity attacks, unauthorised access), RAG pipeline (retrieval manipulation), embedding generation process
LLM09	Misinformation	LLM generates credible-sounding but false content	LLM inference endpoint (hallucination), vector database (stale or incorrect source documents), any user-facing output channel
LLM10	Unbounded Consumption	Uncontrolled resource usage leading to DoS or financial exploitation	LLM inference endpoint (expensive queries, denial of wallet), API gateway (insufficient rate limiting), training pipeline (resource-heavy junk data injection)

This table is designed to work in two directions:

Risk → Component: *"Prompt injection, where does it live?"* Look at the row. It primarily targets the inference endpoint and the RAG pipeline. Those are the components that need input validation and prompt boundary enforcement.

Component → Risk: *"We're deploying a vector database for RAG, what risks does it carry?"* Scan the "Where It Lives" column. The vector database appears under

LLM01 (indirect prompt injection), LLM08 (embedding weaknesses), and LLM09 (misinformation from stale sources). That's your assessment scope for that component.

Here are the risk profiles for the three most critical components:

LLM Inference Endpoint carries the highest risk concentration. It appears in **seven** of the ten OWASP entries: **LLM01 (prompt injection), LLM02 (sensitive info disclosure), LLM05 (improper output handling), LLM06 (excessive agency), LLM07 (system prompt leakage), LLM09 (misinformation), and LLM10 (unbounded consumption)**. This is the component that requires the most comprehensive hardening.

Vector Database / RAG Pipeline appears in **three** entries: **LLM01 (indirect prompt injection via retrieved content), LLM08 (embedding weaknesses), and LLM09 (misinformation from stale or incorrect source documents)**. Hardening focuses on input validation for indexed content, access controls on the vector store, and freshness monitoring for source documents.

Training Pipeline is the primary component for data and model supply chain threats (LLM03). It appears in three entries: **LLM02 (sensitive data entering training), LLM03 (third-party datasets, compromised base models, poisoned fine-tuning data), and LLM04 (data and model poisoning)**. Note that LLM03 also affects plugin or tool integrations via compromised dependencies, but the training pipeline is where third-party models and datasets enter the system through most directly.

Connecting OWASP Back to STRIDE and ATLAS

OWASP, STRIDE and ATLAS aren't competing frameworks, they are layers of the same assessment:

Layer	What It Does	When You Use It
STRIDE-AI	Categorises threats by type	Initial threat identification, "what could go wrong"
MITRE ATLAS	Documents specific attack techniques	Enrichment, "how exactly would an attacker do this"
OWAPS LLM Top 10	Maps risks to components and prioritises	Assessment and scoping, "where does this risk live and how critical is it"

Think of it as zoom levels. STRIDE gives you the wide-angle view. ATLAS gives you the technical detail. OWASP tells you **where to point the camera**.

How many of the OWASP LLM Top 10 entries affect the LLM Inference Endpoint?

6

An organisation notices their chatbot is rendering LLM output directly in the browser without sanitisation. Which OWASP entry does this fall under?

Improper Output Handling

Which component in a typical LLM architecture is the primary one that needs hardening against data and model supply chain risks (LLM03)?

Training Pipeline

Task 7: OWASP LLM Top 10: Mapping Risks to Components

You'll be selecting OWASP LLM Top 10 vulnerabilities, mapping them to architecture components, and justifying your choices to put your threat modelling instincts to the test. This task can be used to practice your knowledge of AI systems and threats.

OWASP LLM Top 10 — Threat Assessment

Map risks to MegaCorp's AI chatbot architecture

Phase 1 — Selection

Phase 1/3 ⓘ Show Instructions ⌵ Exit Fullscreen ↺ Restart

LLM01

LLM02

LLM03

LLM04

LLM05

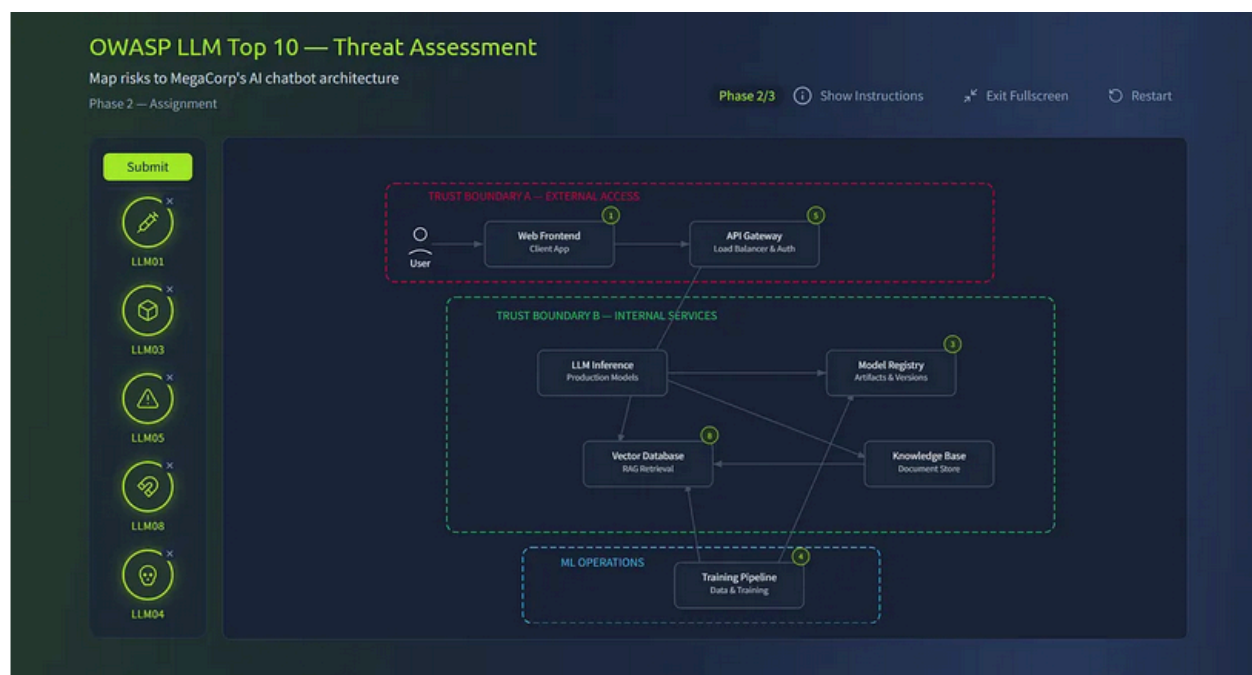
LLM06

Selected Vulnerabilities

5 / 5 selected

Confirm Selection

- Prompt Injection**
Malicious instructions injected to hijack model behaviour
- Supply Chain Vulnerabilities**
Compromised packages, models or datasets entering the pipeline
- Improper Output Handling**
Unvalidated LLM output injected into downstream systems
- Vector & Embedding Weaknesses**
Adversarial embeddings corrupt RAG retrieval
- Data & Model Poisoning**
Corrupted training data poisons model behaviour



OWASP LLM Top 10 — Threat Assessment

Map risks to MegaCorp's AI chatbot architecture

Phase 3 — Assessment

Phase 3/3 ⓘ Show Instructions ⌵ Exit Fullscreen ⌂ Restart

LLM01

LLM05

LLM04

LLM03

LLM08

✎ LLM01: Prompt Injection

Mapped to: Web Frontend

Why does this vulnerability apply here?

The frontend stores model weights and can be poisoned directly through the UI.

The frontend executes SQL queries that the LLM rewrites via injection.

User input fields on the frontend are the entry point for injected prompts before any server-side validation occurs.

Criticality

Low Medium High

Cancel Save

75

/ 75 pts

PASSED

OWASP LLM Top 10 — Threat Assessment

Map risks to MegaCorp's AI chatbot architecture

Phase 3 — Assessment

Phase 3/3 ⓘ Show Instructions ⌵ Exit Fullscreen ⌂ Restart

LLM01

LLM05

LLM04

LLM03

LLM08

⚠ LLM05: Improper Output Handling

Mapped to: API Gateway

Why does this vulnerability apply here?

The API gateway is vulnerable because it trains a shadow model on the raw output stream.

The gateway is responsible for sanitising and validating model output before passing it downstream — failure here lets unsafe content propagate to every consuming service.

The API gateway fails by storing unencrypted output in the model registry.

Criticality

Low Medium High

Cancel Save

75

/ 75 pts

PASSED

AI Security 2.3. AI Threat Modelling

23

OWASP LLM Top 10 — Threat Assessment

Map risks to MegaCorp's AI chatbot architecture

Phase 3 — Assessment

Phase 3/3 ⓘ Show Instructions ⌵ Exit Fullscreen ⌂ Restart

LLM01

LLM05

LLM04

LLM03

LLM08

LLM04: Data & Model Poisoning

Mapped to: Training Pipeline

Why does this vulnerability apply here?

The training pipeline is poisoned when users submit too many requests and exhaust its memory.

Poisoned training data fed into the pipeline causes the model to learn manipulated behaviours or backdoors that persist after deployment.

The training pipeline is attacked by intercepting HTTPS traffic between the frontend and the API.

Criticality

Low Medium **High**

Cancel Save

75 / 75 pts

PASSED

OWASP LLM Top 10 — Threat Assessment

Map risks to MegaCorp's AI chatbot architecture

Phase 3 — Assessment

Phase 3/3 ⓘ Show Instructions ⌵ Exit Fullscreen ⌂ Restart

LLM01

LLM05

LLM04

LLM03

LLM08

LLM03: Supply Chain Vulnerabilities

Mapped to: Model Registry

Why does this vulnerability apply here?

The model registry executes user prompts to generate new model versions dynamically.

The model registry is vulnerable because it stores plaintext user passwords alongside model files.

A compromised or backdoored model uploaded to the registry gets served directly to production without integrity verification.

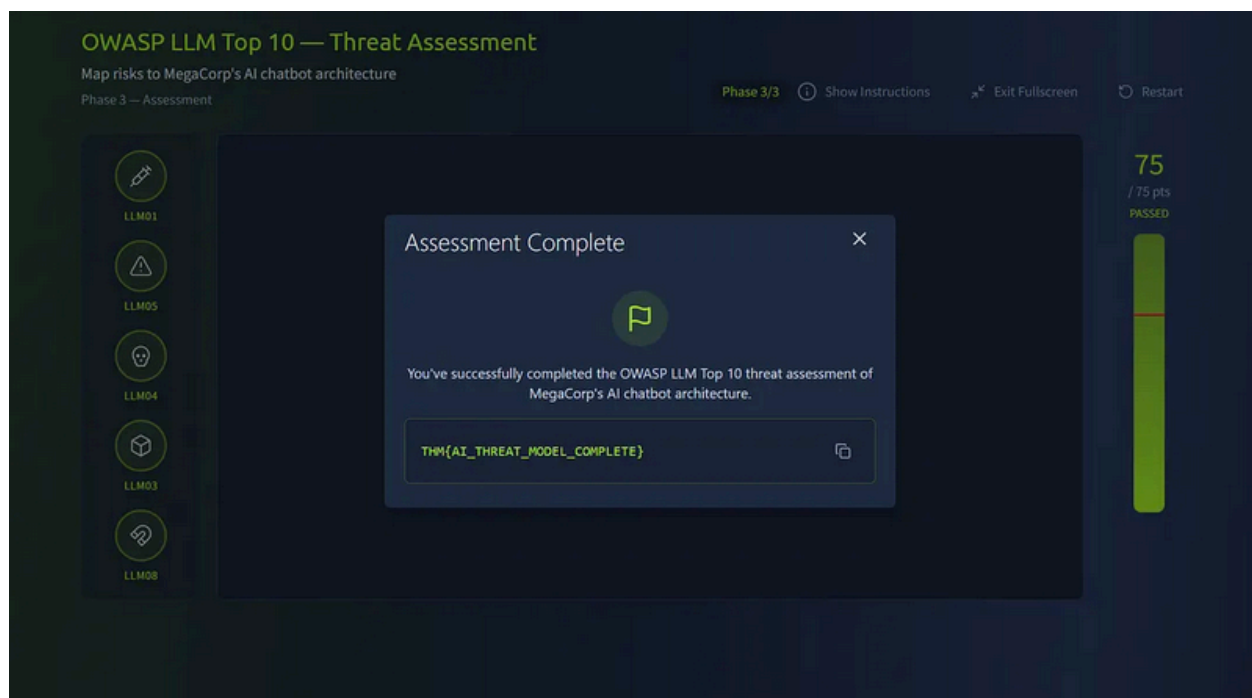
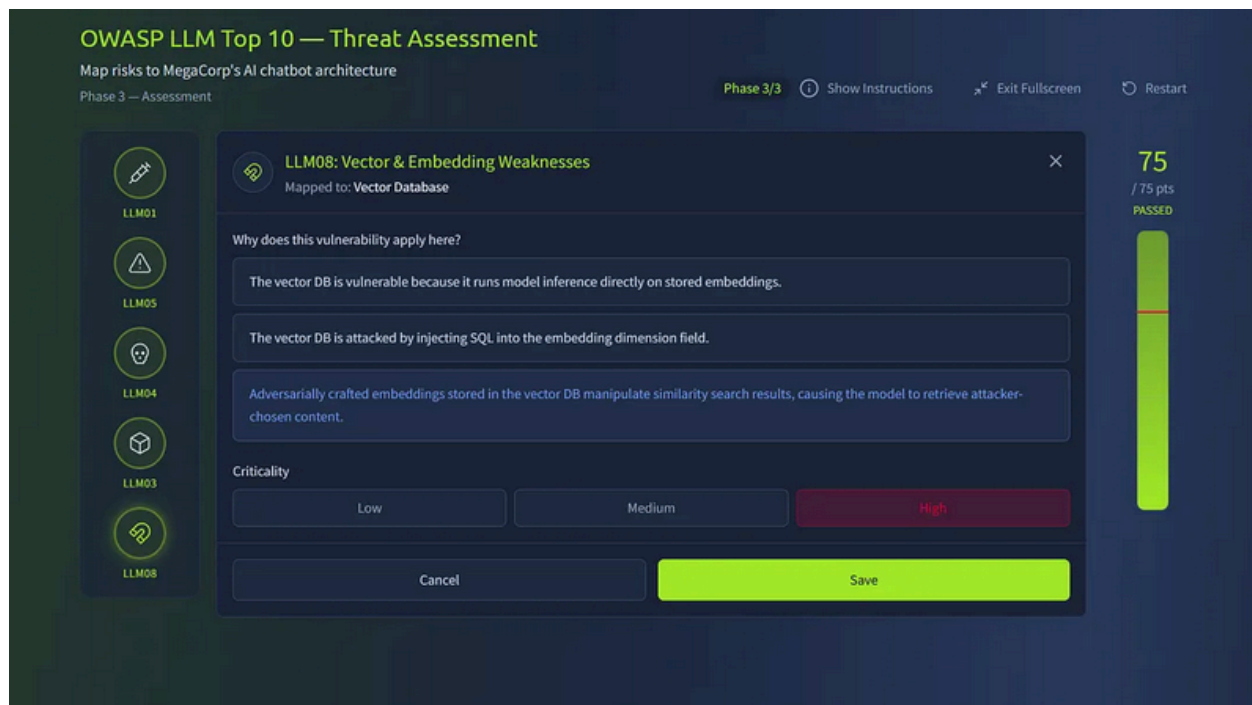
Criticality

Low Medium **High**

Cancel Save

75 / 75 pts

PASSED

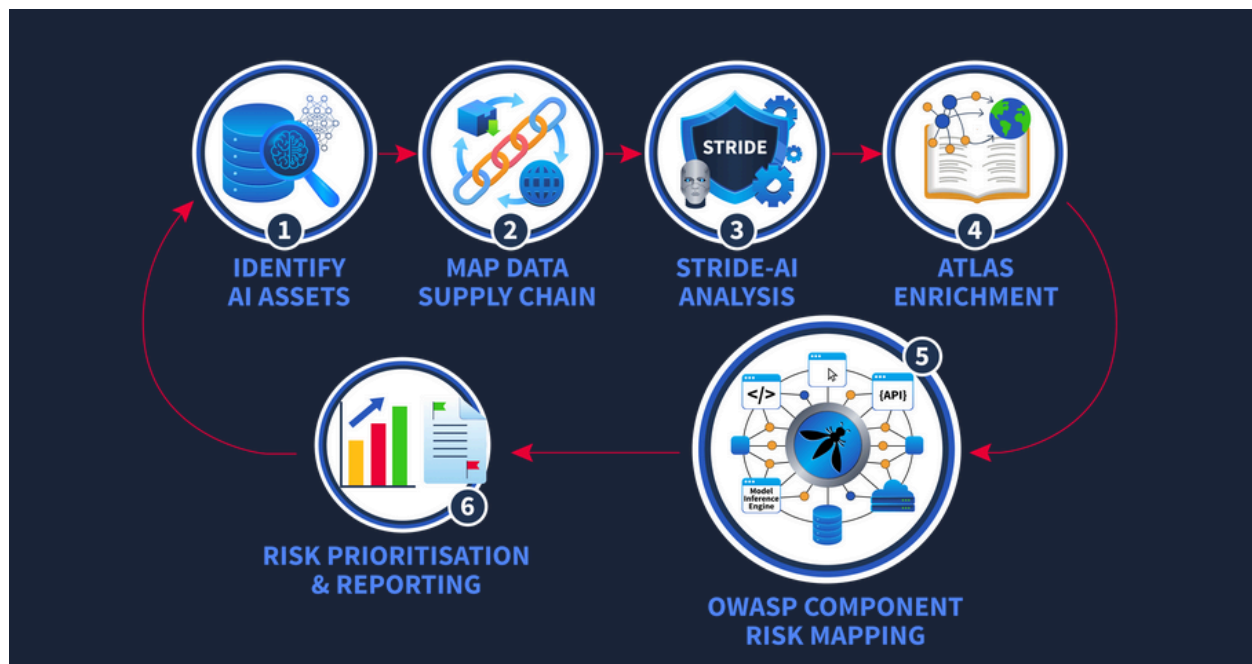


THM{AI_THREAT_MODEL_COMPLETE}

Task 8: Conclusion

- **Identified AI-specific assets** training data, model weights, embeddings, system prompts, feature stores, and model registries, that expand the attack surface beyond traditional applications
- **Mapped the AI data supply chain** understanding how data flows from collection through training to inference, and where each stage is vulnerable to compromise
- **Adapted STRIDE for AI systems**, applying the six familiar threat categories with AI-specific context, from data poisoning under Tampering to jailbreaking under Elevation of Privilege
- **Enriched findings with MITRE ATLAS** using the AI-specific technique catalogue to move from general threat categories to documented attack methods with technique IDs and mitigations
- **Mapped risks to components using the OWASP LLM Top 10** the primary assessment lens that lets you look at an architecture diagram and immediately identify which components carry which risks and at what severity
- **Applied everything to MegaCorp** assessing real components, mapping OWASP risks with STRIDE and ATLAS enrichment, and generating a prioritised threat assessment

The Workflow at a GlanceS



This workflow is repeatable. Every time your organisation deploys a new AI system, updates a model, or introduces agentic capabilities, you can run the same process. The frameworks evolve, ATLAS adds new techniques, OWASP updates its list, but the methodology stays consistent.

Key Takeaways:

AI systems aren't just traditional applications with a model bolted on. They have different assets, a separate data supply chain, and failure modes that require adapted approaches.

STRIDE gives you the threat categories, ATLAS gives you the techniques.

Together, they provide the vocabulary; STRIDE tells you *what type* of threat you're looking at; ATLAS tells you *exactly how an attacker* would execute it and what mitigations to apply.

OWASP tells you where to point the camera. The LLM Top 10 is the framework that maps risks directly to architectural components. It's what lets you look at a deployment and say *"this component carries these risks at this severity"*, and that's the skill that makes a threat assessment actionable.

- **MITRE ATLAS:** Explore the full technique catalogue, case studies, and mitigations beyond what we covered here

- **OWASP AI Exchange:** Explore broader AI security guidance including agentic AI and non-LLM systems